

Aufgabenblatt: Verzweigungen

Aufgabe 1: Altersgruppe

Beschreibung: Viele Programme reagieren auf Zahlenwerte unterschiedlich, je nachdem in welchem Bereich sie liegen. Eine `if-else if-else`-Kette bildet jeden Bereich auf eine eigene Aktion ab und stellt sicher, dass genau eine Verzweigung ausgeführt wird.

Aufgabenstellung: Lies ein Alter als Ganzzahl ein und gib je nach Altersgruppe eine passende Meldung aus.

Bereich	Meldung
Unter 18	Du bist minderjaehrig.
18 bis 64	Du bist volljaehrig.
Ab 65	Du bist Rentner.

```
Bitte gib dein Alter ein: 70
Du bist Rentner.
```

Aufgabe 2: Gerade oder ungerade

Beschreibung: Der Modulo-Operator in Kombination mit einer Bedingung ist eines der haeufigsten Muster in der Programmierung. Ob eine Zahl durch 2 teilbar ist, laesst sich direkt aus dem Rest der Division ablesen.

Aufgabenstellung: Lies eine ganze Zahl ein. Pruefe mit dem Modulo-Operator, ob sie durch 2 teilbar ist, und gib eine entsprechende Meldung aus.

```
Bitte gib eine Zahl ein: 7
ungerade Zahl
```

Aufgabe 3: Kinokontrolle

Beschreibung: Zugangsregeln bestehen in der Praxis meist aus mehreren verknuepften Bedingungen. Logische UND- und ODER-Operatoren ermoeeglichen es, solche Regeln praezise und lesbar direkt im Code auszudruecken.

Aufgabenstellung: Ein Kino zeigt einen Film ab 16. Eintritt erhaelt, wer mindestens 16 Jahre alt ist, oder wer mindestens 12 Jahre alt ist und von einem Erwachsenen begleitet wird. Lies Alter (int) und Begleitung (boolean) ein und gib aus, ob Zutritt erteilt oder verweigert wird.

```
Wie alt bist du? 14
Bist du in Begleitung eines Erwachsenen (true/false)? true
Zutritt erlaubt.
```

Aufgabe 4: Wochentag mit switch

Beschreibung: Switch-Anweisungen eignen sich besonders fuer Situationen, in denen ein Wert gegen eine feste Menge bekannter Faelle geprueft wird. Der Code bleibt dabei uebersichtlicher als eine lange `if-else if` -Kette.

Aufgabenstellung: Lies eine Zahl zwischen 1 und 7 ein. Gib den zugehoerigen Wochentag aus (1 = Montag, 7 = Sonntag). Bei einer ungueltigen Eingabe erscheint eine Fehlermeldung. Verwende eine klassische `switch` -Anweisung mit `case` , `break` und `default` .

```
Bitte gib eine Zahl (1-7) ein: 5
Freitag
```

[Bonus] Kalt oder warm

Beschreibung: Der Ternary-Operator weist einer Variablen einen Wert basierend auf einer Bedingung zu. Durch Verketteten mehrerer Ternary-Ausdruecke lassen sich auch mehr als zwei Faelle und Werte in einem einzigen Ausdruck abdecken.

Aufgabenstellung: Lies eine Temperatur als Ganzzahl ein. Setze eine boolean-Variable `isCold` mit dem Ternary-Operator auf `true` , wenn die Temperatur unter 10 Grad liegt, sonst auf `false` . Gib den Wert von `isCold` sowie eine passende Textnachricht aus. Verwende fuer die Nachricht ebenfalls den Ternary-Operator.

```
Wie viel Grad hat es heute? 8
isCold = true
Es ist kalt draussen!
```

[Bonus] Taschenrechner

Beschreibung: Taschenrechner sind ein klassischer Anwendungsfall fuer `switch`: Jede Rechenoperation ist ein eigener Fall, unbekannte Eingaben landen im `default` -Zweig. Sonderfaelle wie Division durch null muessen zusaetzlich mit

einer Bedingung abgefangen werden. Diese Aufgabe wird in Einheit 05 (Schleifen) und Einheit 08 (Methoden) schrittweise erweitert.

Aufgabenstellung: Lies zwei Zahlen (double) und ein Rechenzeichen (+ , - , * , /) mit dem Scanner ein. Das Zeichen wird als char eingelesen mit scanner.next().charAt(0) . Berechne das Ergebnis mit einer switch - Anweisung und gib es aus. Bei Division durch 0 oder unbekannter Operation erscheint eine Fehlermeldung.

```
Erste Zahl: 10.0
Operation (+, -, *, /): /
Zweite Zahl: 4.0
10.0 / 4.0 = 2.5
```

```
Erste Zahl: 5.0
Operation (+, -, *, /): /
Zweite Zahl: 0.0
Fehler: Division durch 0 ist nicht erlaubt.
```

```
Erste Zahl: 3.0
Operation (+, -, *, /): %
Unbekannte Operation: %
```